VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"Jnana Sangama", Belagavi - 590018, Karnataka

PARALLEL COMPUTING (BCS702)

Report on

Parallel Image Edge Detection Using OpenMP And CUDA

Submitted in partial fulfilment for the requirements for the 7th Semester

INFORMATION SCIENCE AND ENGINEERING

Submitted by

NAME	USN
DHRUV AHUJA	1BI23IS034
GAUTAM SHARMA	1BI23IS038
J CHETAN RAJ	1BI23IS048

Under the Guidance of

Prameela R

Assistant Professor

Dept. of ISE, BIT, BANGALORE

BANGALORE INSTITUTE OF TECHNOLOGY

K. R. Road, V. V. Pura, Bengaluru – 560004

Department of Information Science Engineering

2026 – 2027

ABSTRACT

Edge detection is one of the most fundamental operations in image processing and computer vision, and it is used in autonomous vehicles, medical imaging, document scanning and video analytics. Detecting edges requires every pixel of an image to be processed with a small neighbourhood filter, so the amount of computation grows directly with image resolution. A single 8K frame contains 33.2 million pixels, and a traditional sequential implementation needs more than 120 milliseconds to process it, which is far too slow for real-time video.

This project, titled **"Parallel Image Edge Detection Using OpenMP and CUDA,"** implements a two-stage edge-detection pipeline, a 5×5 Gaussian blur followed by a 3×3 Sobel filter, in four ways: a sequential C baseline, a shared-memory parallel version using OpenMP on a multi-core CPU, and two GPU versions using NVIDIA CUDA (a naive global-memory kernel and an optimised shared-memory tiled kernel). All versions share the same per-pixel arithmetic, so their outputs can be compared pixel by pixel.

The system measures the execution time of every version on four image sizes, from HD (0.9 million pixels) to 8K (33.2 million pixels), and evaluates speedup, parallel efficiency, the effect of OpenMP scheduling policies, Amdahl's law through the Karp–Flatt metric, and the cost of CPU–GPU data transfer. On an Apple M5 processor, OpenMP with 10 threads achieved a speedup of 4.2×; on an NVIDIA Tesla T4 GPU, the CUDA kernels were 32× faster than the sequential version, or 6.8× when data transfer time is included.

The results demonstrate both the benefits and the limitations of parallel processing: heterogeneous CPU cores, memory bandwidth, thread overhead and data movement all limit the achievable speedup. Every parallel output was verified to be identical to the sequential output.

TABLE OF CONTENTS

CONTENTS	PAGE NO
ABSTRACT	i
TABLE OF CONTENTS	ii
LIST OF FIGURES	iii
CHAPTER 1: INTRODUCTION	1
CHAPTER 2: PROBLEM STATEMENT	2
CHAPTER 3: PROPOSED SOLUTION	3
CHAPTER 4: OBJECTIVES	4
CHAPTER 5: SOFTWARE AND HARDWARE REQUIREMENTS	5-6
CHAPTER 6: USER FLOW	7-8
CHAPTER 7: SYSTEM ARCHITECTURE	9-10
CHAPTER 8: IMPLEMENTATION	11-21
CHAPTER 9: RESULTS	22-28
CHAPTER 10: ADVANTAGES	29-30
CHAPTER 11: APPLICATIONS	31
CHAPTER 12: CONCLUSION	32
REFERENCES	33

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
6.1	User Flow	7
7.1	System Architecture	9
9.1	Sequential and OpenMP Implementation Output	22
9.2	OpenMP Scheduling and Correctness Output	23
9.3	CUDA Implementation Output	23
9.4	Input Image and Edge-Detection Output (Full HD)	24
9.5	Execution Time Comparison of All Versions	24
9.6	OpenMP Speedup (Static Scheduling)	25
9.7	OpenMP Parallel Efficiency (Static Scheduling)	25
9.8	Static vs Dynamic Scheduling (8K Image)	26
9.9	GPU Time Breakdown – Data Transfer vs Computation	27

CHAPTER 1

INTRODUCTION

Parallel Image Edge Detection Using OpenMP and CUDA applies parallel computing to one of the most widely used operations in image processing. Edge detection identifies the points in an image where brightness changes sharply; these points usually mark the boundaries of objects. It is the first step in many computer-vision systems, including lane detection in self-driving cars, organ and tumour outlining in medical scans, optical character recognition in document scanners, and object tracking in video surveillance.

The most common edge-detection pipeline has two stages. First, a **Gaussian blur** replaces every pixel with a weighted average of its 5×5 neighbourhood, which removes noise. Second, the **Sobel filter** computes the horizontal and vertical brightness gradients (gx and gy) of every pixel from its 3×3 neighbourhood; the edge strength is $\sqrt{(gx^2 + gy^2)}$. Both stages are simple, but they must be applied to every pixel. A single 8K frame (7680 × 4320) contains 33.2 million pixels and needs roughly one billion arithmetic operations, and video requires 30 to 60 such frames every second. In our measurements, a sequential implementation took about 126 milliseconds per 8K frame, only about 8 frames per second.

Edge detection is an ideal problem for parallel computing because each output pixel depends only on a small neighbourhood of the *input* image. No output pixel depends on another output pixel, so all pixels can be computed at the same time. This property is called **data parallelism**, and it maps naturally onto both multi-core CPUs and Graphics Processing Units (GPUs).

The main objective of this project, **"Parallel Image Edge Detection Using OpenMP and CUDA,"** is to implement the same edge-detection pipeline sequentially and in parallel, and to compare their performance. The project uses the C programming language for the sequential baseline, **OpenMP** for shared-memory parallel processing on a multi-core CPU, and **NVIDIA CUDA** for massively parallel processing on a GPU, which is an example of heterogeneous computing. The execution times of all versions are measured on images from HD to 8K, and the results are analysed using speedup, parallel efficiency, scheduling behaviour, Amdahl's law and the cost of moving data between the CPU and the GPU.

CHAPTER 2

PROBLEM STATEMENT

Image processing operations such as edge detection are used in a wide range of real-time applications, from autonomous driving to medical imaging. A traditional sequential implementation processes the pixels of an image one after another on a single CPU core. As image resolution increases from HD to 4K and 8K, the number of pixels, and therefore the computation time, increases proportionally. For high-resolution images and video, sequential processing cannot meet real-time requirements.

The main problem is to reduce the execution time of edge detection on large images by using parallel processing. Since every output pixel can be computed independently, the work can be divided among many processing units. However, parallel execution introduces its own challenges: dividing the work evenly among threads, avoiding race conditions, managing thread start-up and synchronisation overhead, using the memory hierarchy efficiently, and, on a GPU, transferring the image between CPU memory and GPU memory.

This project aims to implement and compare sequential and parallel edge detection using C. OpenMP is used to distribute the image rows among multiple CPU threads in a shared-memory system, and CUDA is used to assign one GPU thread to each pixel. Two CUDA versions are compared: one that reads directly from global memory and one that uses fast on-chip shared memory.

The system measures the execution time of every implementation for different image sizes, verifies that every parallel version produces exactly the same output as the sequential version, and analyses the factors that limit the speedup, including heterogeneous CPU cores, memory bandwidth, scheduling, Amdahl's law and CPU–GPU data transfer. The goal is to demonstrate clearly how, and how much, parallel computing improves the performance of a real image-processing workload.

CHAPTER 3

PROPOSED SOLUTION

The proposed solution implements a two-stage edge-detection pipeline, a 5×5 Gaussian blur followed by a 3×3 Sobel filter, and executes it with four implementations that share exactly the same per-pixel arithmetic. Because the arithmetic is shared (integer weights for the blur and a correctly rounded square root for the edge magnitude), every implementation must produce a bit-identical output image, which makes correctness easy to verify.

In the **sequential implementation**, a single CPU core processes the image row by row and pixel by pixel. This version is the baseline against which every speedup is measured.

In the **OpenMP implementation**, the outer loop over image rows is parallelised with the directive `#pragma omp parallel for schedule(runtime)`. The rows are divided among a team of threads that run concurrently on the cores of the CPU and share the input and output images in memory. Because every thread writes different output pixels and only reads the input, no locks, critical sections or reductions are needed. The scheduling policy (static or dynamic) can be selected at run time through the OMP_SCHEDULE environment variable, which allows the effect of scheduling to be studied without recompiling.

In the **CUDA implementation**, the image is copied to GPU memory and processed by thousands of GPU threads, one per pixel, organised as a two-dimensional grid of 16×16-thread blocks. In the *naive* kernels every thread reads all of its neighbours directly from global memory. In the *shared-memory (tiled)* kernels each block first copies its tile of the image, together with a border of neighbouring pixels called the halo, into fast on-chip shared memory, synchronises with `__syncthreads()`, and then computes from the shared copy. The blur and the Sobel filter run as two separate kernels, because the Sobel stage needs the completed blurred image.

The system measures the execution time of every implementation on four image sizes (HD, Full HD, 4K and 8K): CPU code is timed with `clock_gettime`, and GPU code with CUDA events, which time the data transfers and the kernels separately. Every output is compared with the sequential output pixel by pixel. Finally, the results are collected in CSV files and plotted as charts of execution time, speedup, efficiency, scheduling behaviour and GPU time breakdown.

The proposed solution provides a clear and practical way to understand shared-memory parallelism, GPU programming and performance analysis on a real image-processing workload.

CHAPTER 4

OBJECTIVES

The main objective of this project is to implement image edge detection using OpenMP and CUDA and to compare their performance with a sequential implementation. The project aims to understand how parallel computing techniques improve computational efficiency by distributing pixel computations among multiple CPU threads and GPU threads.

The specific objectives of the project are:

1. **To understand image edge detection:** Study the Gaussian blur and Sobel operators, how they detect edges, and why each output pixel can be computed independently.
2. **To implement sequential edge detection:** Develop a sequential C implementation of the blur and Sobel pipeline and use it as the baseline for performance comparison and correctness checking.
3. **To implement parallel computation using OpenMP:** Use OpenMP directives to divide the image rows among multiple threads on a shared-memory multi-core CPU, with correct variable scoping and no race conditions.
4. **To implement parallel computation using CUDA:** Use CUDA to assign one GPU thread per pixel, and compare a naive global-memory kernel with an optimised shared-memory tiled kernel.
5. **To analyse performance:** Measure speedup and parallel efficiency, compare static and dynamic scheduling, estimate the serial fraction using Amdahl's law and the Karp–Flatt metric, and measure the cost of CPU–GPU data transfer.
6. **To verify correctness:** Confirm that every parallel implementation produces an output image identical to the sequential output, pixel by pixel.
7. **To demonstrate practical applications:** Explain how parallel image processing is applied in autonomous vehicles, medical imaging, video analytics, document processing and other real-world systems.

CHAPTER 5

SOFTWARE AND HARDWARE REQUIREMENTS

1. SOFTWARE REQUIREMENTS

1.1 Operating System

- macOS 12 or later (Apple Silicon or Intel)
- Ubuntu 20.04 or later (64-bit)
- Windows 10/11 (64-bit) with Windows Subsystem for Linux (WSL)
- Google Colab (Linux runtime with an NVIDIA GPU) for the CUDA version

1.2 Compilers and Tools

- GCC 9 or newer, or Clang, with OpenMP support (on macOS: Apple Clang with the Homebrew libomp runtime)
- NVIDIA CUDA Toolkit 11 or newer, including the nvcc compiler
- GNU Make, to build all versions with a single command
- Bash shell, to run the benchmark and demo scripts
- Visual Studio Code or any text editor for code development
- Git, for version control

1.3 Python Dependencies

Python is used only for preparing test images, verifying outputs and plotting results. Install the dependencies using the following command:

```
pip install pillow matplotlib
```

Main Dependency Groups Include

- Image Handling: Pillow (creating test images, comparing outputs, converting to PNG)
- Data Visualization: Matplotlib (performance comparison charts)
- Data Processing: Python csv module (included in the standard library)

1.4 Optional Development Tools

- Jupyter Notebook or Google Colab, for running the self-contained CUDA notebook
- Image viewer (Preview, Photos or similar) for viewing input and output images

2. HARDWARE REQUIREMENTS

2.1 Processor

- Minimum: Dual-core 64-bit processor

- Recommended: 8 or more cores for meaningful OpenMP scaling (tested on Apple M5, 10 cores: 4 performance + 6 efficiency)

2.2 Graphics Processor (for CUDA)

- Any NVIDIA GPU with CUDA support (compute capability 7.5 or newer recommended)
- Tested on NVIDIA Tesla T4 (Google Colab): 40 streaming multiprocessors, 2,560 CUDA cores, 16 GB memory

2.3 Memory (RAM)

- Minimum: 4 GB
- Recommended: 8 GB or more (an 8K grayscale image occupies 33 MB, and three image buffers are used)

2.4 Storage

- Minimum: 500 MB of free disk space for the test images, outputs and results

2.5 Network Requirements

- Internet connection for installing packages and for accessing Google Colab
- No internet connection is required to run the CPU versions after installation

CHAPTER 6

USER FLOW

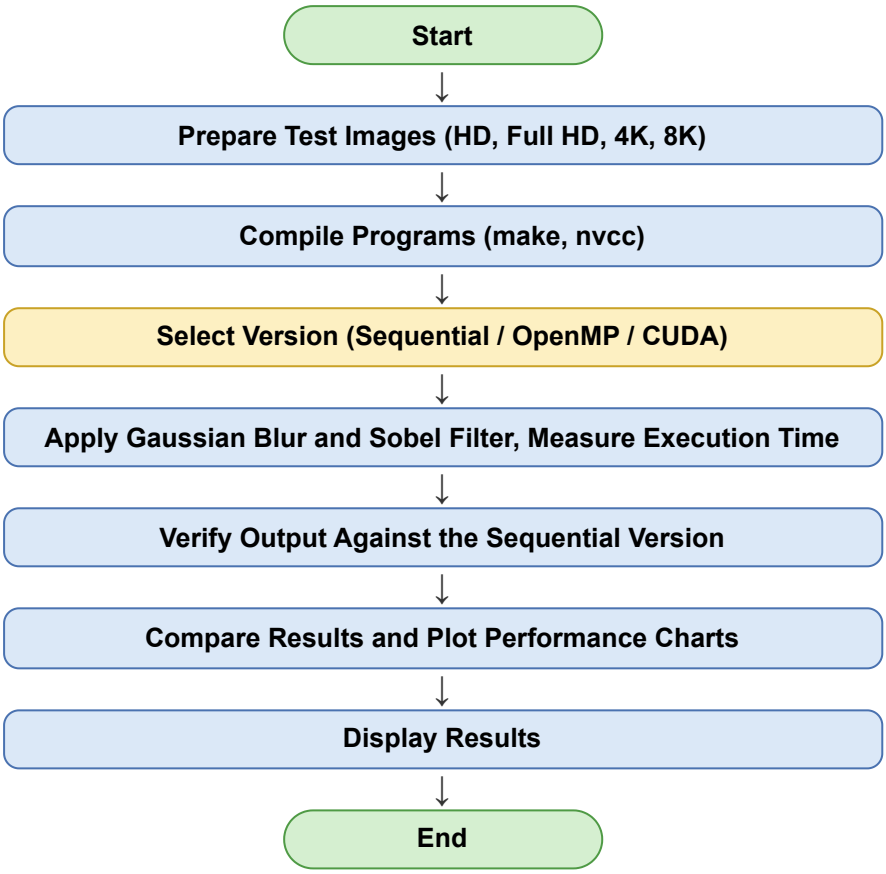


Figure 6.1: User Flow

Figure 6.1 describes the user flow of the Parallel Image Edge Detection system. The process begins when the user prepares the test images: the script `tools.py prepare` creates grayscale images in four sizes, HD (1280×720), Full HD (1920×1080), 4K (3840×2160) and 8K (7680×4320), either from a photograph supplied by the user or from a generated test picture. The user then compiles the programs with `make` for the CPU versions and `nvcc` for the CUDA version.

Next, the user selects the implementation to run: sequential, OpenMP or CUDA. For OpenMP, the user can also choose the number of threads (OMP_NUM_THREADS) and the scheduling policy (OMP_SCHEDULE); for CUDA, the user can choose between the naive and shared-memory kernels and the block size. Each program reads the input image, applies the Gaussian blur and the Sobel filter, records the execution time, and writes the edge map as an output image.

After the computation, the system verifies that every parallel output is identical to the sequential output. The benchmark script runs every version on every image size and stores the timings in CSV files, and the plotting script turns them into charts of execution time, speedup, efficiency, scheduling behaviour and GPU time breakdown. Finally, the user reviews the output images and the performance results to understand the differences between sequential and parallel processing.

CHAPTER 7

SYSTEM ARCHITECTURE

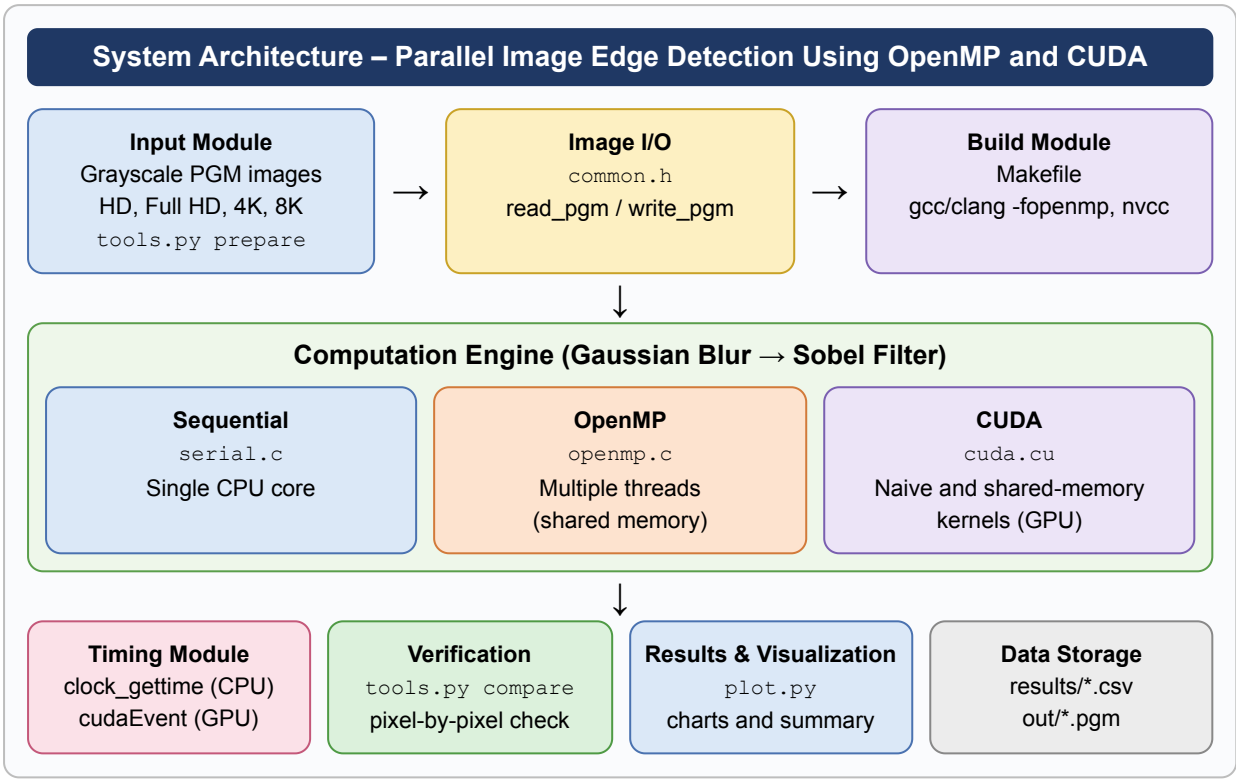


Figure 7.1: System Architecture

Figure 7.1 represents the overall structure of the Parallel Image Edge Detection system. The process begins with the **input module**, which prepares grayscale test images in the PGM format at four resolutions. PGM is the simplest image format: a short text header giving the width, height and maximum value, followed by one byte per pixel. This keeps the C programs free of external image libraries.

The **image I/O module** (`common.h`) reads and writes PGM files and also contains the shared per-pixel arithmetic: the 5×5 Gaussian weights, the Sobel gradients and the edge-strength function. The same header is compiled by the C compiler for the CPU versions and by nvcc for the CUDA version, which guarantees that all versions perform identical arithmetic. The **build module** (Makefile) compiles the sequential and OpenMP versions with GCC or Clang and the CUDA version with nvcc.

The **computation engine** is the main processing component. It supports three approaches. In sequential processing, a single core applies the blur and the Sobel filter to every pixel in turn. In OpenMP parallel processing, the image rows are divided among multiple threads that share the image in memory. In CUDA parallel processing, the image is copied to the GPU, where one

thread per pixel computes the result using either global memory (naive kernel) or on-chip shared memory (tiled kernel), and the output is copied back.

The **timing module** records the execution time of each stage. CPU versions use `clock_gettime` and report the best of several runs; the CUDA version uses CUDA events to time the host-to-device copy, the blur kernel, the Sobel kernel and the device-to-host copy separately. The **verification module** compares every parallel output with the sequential output pixel by pixel. The **results and visualization module** produces charts and a summary table, and the **data storage module** saves the timings as CSV files and the output images as PGM files for later analysis.

Therefore, Figure 7.1 illustrates the complete workflow of the system: images are prepared and loaded, processed by the sequential or parallel computation engine, timed and verified, and finally presented as output images, tables and charts that demonstrate the principles and practical benefits of parallel computing with OpenMP and CUDA.

CHAPTER 8

IMPLEMENTATION

8.1 Common Module: Image I/O and Filter Arithmetic

The common module is a C header shared by all versions. It reads and writes grayscale PGM images, provides a millisecond timer, prints one line of comma-separated results per run, and defines the per-pixel filter arithmetic. The Gaussian blur uses the integer weights [1 4 6 4 1] × [1 4 6 4 1], which sum to 256, and pixels outside the image are replaced by the nearest edge pixel ("clamp to edge"). The functions marked `HD` are compiled for both the CPU and the GPU when the header is compiled by nvcc, so the edge-strength calculation is identical on both.

Code: common.h

```
/*
 * common.h - shared helpers for all three versions (serial, OpenMP, CUDA).
 *
 *   - read/write grayscale PGM images (the simplest image format: a tiny text
 *     header followed by raw bytes, one byte per pixel)
 *   - a millisecond timer
 *   - the per-pixel filter math used by the serial and OpenMP versions
 *
 * It compiles as C (gcc/clang) and as C++ (nvcc), so the CUDA version can
 * reuse the image I/O and the edge-strength formula.
 */
#ifndef COMMON_H
#define COMMON_H

#include <ctype.h>
#include <math.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

/* Functions marked HD run on both the CPU and the GPU when compiled by nvcc. */
#ifdef __CUDACC__
#define HD __host__ __device__
#else
#define HD
#endif

typedef struct {
    int width;
    int height;
    uint8_t *data; /* width * height bytes, row by row */
} Image;

static inline void die(const char *msg) {
    fprintf(stderr, "error: %s\n", msg);
    exit(1);
}

static inline void *xmalloc(size_t n) {
    void *p = malloc(n);
    if (!p) die("out of memory");
    return p;
}
```

```
/* ---------------------------------------------------------------- PGM I/O */

static inline void pgm_skip(FILE *f) { /* skip whitespace and # comments */
    int c;
    while ((c = fgetc(f)) != EOF) {
        if (c == '#') {
            while ((c = fgetc(f)) != EOF && c != '\n') {}
        } else if (!isspace(c)) {
            ungetc(c, f);
            return;
        }
    }
}

static inline int pgm_int(FILE *f) {
    int v;
    pgm_skip(f);
    if (fscanf(f, "%d", &v) != 1) die("malformed PGM header");
    return v;
}

static inline Image read_pgm(const char *path) {
    FILE *f = fopen(path, "rb");
    if (!f) { perror(path); exit(1); }

    char magic[3] = {0};
    if (fread(magic, 1, 2, f) != 2 || strcmp(magic, "P5") != 0)
        die("input must be a binary grayscale PGM (P5) - use: python3 tools.py prepare");

    Image img;
    img.width = pgm_int(f);
    img.height = pgm_int(f);
    int maxval = pgm_int(f);
    if (img.width <= 0 || img.height <= 0 || maxval != 255)
        die("unsupported PGM (need 8-bit grayscale)");
    fgetc(f); /* exactly one whitespace byte separates header and pixels */

    size_t n = (size_t)img.width * img.height;
    img.data = (uint8_t *)xmalloc(n);
    if (fread(img.data, 1, n, f) != n) die("PGM file is truncated");
    fclose(f);
    return img;
}

static inline void write_pgm(const char *path, const uint8_t *data, int w, int h) {
    FILE *f = fopen(path, "wb");
    if (!f) { perror(path); exit(1); }
    fprintf(f, "P5\n%d %d\n255\n", w, h);
    fwrite(data, 1, (size_t)w * h, f);
    fclose(f);
}

/* ---------------------------------------------------------------- timing / CSV */

static inline double now_ms(void) {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec * 1e3 + ts.tv_nsec / 1e6;
}

/* Every program prints exactly one CSV line to stdout; bench.sh adds the header:
 * version,threads,width,height,blur_ms,sobel_ms,transfer_ms,total_ms
 * (for CUDA, "threads" = threads per block and transfer_ms = CPU<->GPU copies) */
static inline void print_result(const char *version, int threads, int w, int h,
                                double blur_ms, double sobel_ms, double transfer_ms) {
    printf("%s,%d,%d,%d,%.3f,%.3f,%.3f,%.3f\n", version, threads, w, h,
           blur_ms, sobel_ms, transfer_ms, blur_ms + sobel_ms + transfer_ms);
    fflush(stdout);
```

```
}

/* ------------------------------------------------------------ filter math */

static inline HD int clampi(int v, int lo, int hi) {
    return v < lo ? lo : (v > hi ? hi : v);
}

/* Edge strength from the Sobel gradients: sqrt(gx^2 + gy^2), capped at 255.
 * Shared with the GPU so every version produces bit-identical output. */
static inline HD uint8_t edge_value(int gx, int gy) {
    float mag = sqrtf((float)(gx * gx + gy * gy));
    return (uint8_t)(mag > 255.0f ? 255.0f : mag);
}

/* 5x5 Gaussian blur weights = [1 4 6 4 1] x [1 4 6 4 1]; they add up to 256. */
static const int GAUSS[5] = {1, 4, 6, 4, 1};

/* Blurred value of pixel (x, y). Pixels outside the image use the nearest
 * edge pixel ("clamp to edge"). */
static inline uint8_t blur_pixel(const uint8_t *in, int w, int h, int x, int y) {
    int sum = 0;
    for (int dy = -2; dy <= 2; dy++) {
        const uint8_t *row = in + (size_t)clampi(y + dy, 0, h - 1) * w;
        for (int dx = -2; dx <= 2; dx++)
            sum += GAUSS[dy + 2] * GAUSS[dx + 2] * row[clampi(x + dx, 0, w - 1)];
    }
    return (uint8_t)((sum + 128) >> 8); /* divide by 256, rounded */
}

/* Sobel edge strength of pixel (x, y) from its 3x3 neighbourhood:
 *     p00 p01 p02
 *     p10  .  p12
 *     p20 p21 p22                                                    */
static inline uint8_t sobel_pixel(const uint8_t *in, int w, int h, int x, int y) {
    const uint8_t *up = in + (size_t)clampi(y - 1, 0, h - 1) * w;
    const uint8_t *mid = in + (size_t)y * w;
    const uint8_t *down = in + (size_t)clampi(y + 1, 0, h - 1) * w;
    int xl = clampi(x - 1, 0, w - 1), xr = clampi(x + 1, 0, w - 1);

    int p00 = up[xl], p01 = up[x], p02 = up[xr];
    int p10 = mid[xl], p12 = mid[xr];
    int p20 = down[xl], p21 = down[x], p22 = down[xr];

    int gx = (p02 + 2 * p12 + p22) - (p00 + 2 * p10 + p20); /* left vs right */
    int gy = (p20 + 2 * p21 + p22) - (p00 + 2 * p01 + p02); /* top vs bottom */
    return edge_value(gx, gy);
}

#endif /* COMMON_H */
```

8.2 Sequential Implementation

The sequential implementation applies the Gaussian blur and then the Sobel filter to every pixel using two nested loops, on a single CPU core. Each stage is run several times and the fastest run is reported, because the first run also pays for the operating system providing fresh memory pages. This version is the baseline for every speedup in the project.

Code: serial.c

```
/*
 * serial.c - the baseline: Gaussian blur, then Sobel edge detection,
 * one pixel at a time on a single CPU core.
```

```
 *
 * usage: ./serial input.pgm output.pgm [repeats]
 */
#include "common.h"

static void gaussian_blur(const uint8_t *in, uint8_t *out, int w, int h) {
    for (int y = 0; y < h; y++)
        for (int x = 0; x < w; x++)
            out[(size_t)y * w + x] = blur_pixel(in, w, h, x, y);
}

static void sobel(const uint8_t *in, uint8_t *out, int w, int h) {
    for (int y = 0; y < h; y++)
        for (int x = 0; x < w; x++)
            out[(size_t)y * w + x] = sobel_pixel(in, w, h, x, y);
}

int main(int argc, char **argv) {
    if (argc < 3) {
        fprintf(stderr, "usage: %s input.pgm output.pgm [repeats]\n", argv[0]);
        return 1;
    }
    int reps = argc > 3 ? atoi(argv[3]) : 5;
    if (reps < 1) reps = 1;

    Image img = read_pgm(argv[1]);
    int w = img.width, h = img.height;
    uint8_t *blurred = (uint8_t *)xmalloc((size_t)w * h);
    uint8_t *edges = (uint8_t *)xmalloc((size_t)w * h);

    /* Run several times and keep the fastest run (the first one also pays
     * for the OS handing us fresh memory pages). */
    double best_blur = 0, best_sobel = 0, best_total = 1e30;
    for (int r = 0; r < reps; r++) {
        double t0 = now_ms();
        gaussian_blur(img.data, blurred, w, h);
        double t1 = now_ms();
        sobel(blurred, edges, w, h);
        double t2 = now_ms();
        if (t2 - t0 < best_total) {
            best_total = t2 - t0;
            best_blur = t1 - t0;
            best_sobel = t2 - t1;
        }
    }

    write_pgm(argv[2], edges, w, h);
    print_result("serial", 1, w, h, best_blur, best_sobel, 0.0);

    free(img.data);
    free(blurred);
    free(edges);
    return 0;
}
```

8.3 OpenMP Implementation

The OpenMP implementation is identical to the sequential version except for one directive above each outer loop: `#pragma omp parallel for schedule(runtime)`. This directive creates a team of threads and divides the iterations of the row loop among them. The loop counters y and x are private to each thread, while the image arrays and dimensions are shared. Because the input image is only read and every thread writes different output pixels, there is no

race condition and no lock, critical section or reduction is required. The `schedule(runtime)` clause allows the scheduling policy to be selected with the OMP_SCHEDULE environment variable, and the program reports which policy was used.

Code: openmp.c

```
/*
 * openmp.c - the same filters, with the image rows shared out among CPU threads.
 *
 * The only real change from serial.c is one line above each outer loop:
 *
 *     #pragma omp parallel for schedule(runtime)
 *
 * Variable scope: the loop counters y and x are private to each thread (every
 * thread has its own copy); the image pointers and sizes are shared (read-only,
 * so no race). Each thread writes different output pixels, so no locks are needed.
 *
 * schedule(runtime) lets us switch between static and dynamic scheduling with
 * the OMP_SCHEDULE environment variable, without recompiling.
 *
 * usage: OMP_NUM_THREADS=4 OMP_SCHEDULE=static ./openmp input.pgm output.pgm [repeats]
 */
#include "common.h"
#ifdef _OPENMP
#include <omp.h>
#endif

static void gaussian_blur(const uint8_t *in, uint8_t *out, int w, int h) {
    #pragma omp parallel for schedule(runtime)
    for (int y = 0; y < h; y++)
        for (int x = 0; x < w; x++)
            out[(size_t)y * w + x] = blur_pixel(in, w, h, x, y);
}

static void sobel(const uint8_t *in, uint8_t *out, int w, int h) {
    #pragma omp parallel for schedule(runtime)
    for (int y = 0; y < h; y++)
        for (int x = 0; x < w; x++)
            out[(size_t)y * w + x] = sobel_pixel(in, w, h, x, y);
}

/* Label for the CSV, e.g. "openmp_static", taken from the active schedule. */
static const char *version_label(void) {
#ifdef _OPENMP
    omp_sched_t kind;
    int chunk;
    omp_get_schedule(&kind, &chunk);
    switch ((int)((unsigned)kind & 0x7fffffffu)) { /* drop the "monotonic" flag bit */
        case omp_sched_static:  return "openmp_static";
        case omp_sched_dynamic: return "openmp_dynamic";
        case omp_sched_guided:  return "openmp_guided";
        default:                return "openmp_auto";
    }
#else
    return "openmp_disabled"; /* compiled without -fopenmp */
#endif
}

int main(int argc, char **argv) {
    if (argc < 3) {
        fprintf(stderr, "usage: %s input.pgm output.pgm [repeats]\n", argv[0]);
        return 1;
    }
    int reps = argc > 3 ? atoi(argv[3]) : 5;
    if (reps < 1) reps = 1;
```

```
#ifdef _OPENMP
    int threads = omp_get_max_threads();
#else
    int threads = 1;
#endif

    Image img = read_pgm(argv[1]);
    int w = img.width, h = img.height;
    uint8_t *blurred = (uint8_t *)xmalloc((size_t)w * h);
    uint8_t *edges = (uint8_t *)xmalloc((size_t)w * h);

    double best_blur = 0, best_sobel = 0, best_total = 1e30;
    for (int r = 0; r < reps; r++) {
        double t0 = now_ms();
        gaussian_blur(img.data, blurred, w, h);
        double t1 = now_ms();
        sobel(blurred, edges, w, h);
        double t2 = now_ms();
        if (t2 - t0 < best_total) {
            best_total = t2 - t0;
            best_blur = t1 - t0;
            best_sobel = t2 - t1;
        }
    }

    write_pgm(argv[2], edges, w, h);
    print_result(version_label(), threads, w, h, best_blur, best_sobel, 0.0);

    free(img.data);
    free(blurred);
    free(edges);
    return 0;
}
```

8.4 CUDA Implementation

The CUDA implementation launches one GPU thread per pixel. Threads are grouped into 16×16 blocks (256 threads, or 8 warps, per block), and the blocks form a two-dimensional grid that covers the whole image; each thread computes its pixel coordinates as `blockIdx × blockDim + threadIdx`. Two versions of each kernel are provided. The **naive** kernels read all neighbours directly from global memory, so each pixel is read by up to 25 different threads. The **tiled** kernels first copy the block's tile and its halo into shared memory cooperatively, wait at `__syncthreads()`, and then compute from the fast on-chip copy. The host code allocates GPU memory, launches the two kernels, and uses CUDA events to time the host-to-device copy, each kernel and the device-to-host copy separately.

Code: cuda.cu

```
/*
 * cuda.cu - the same two filters on an NVIDIA GPU, in two versions:
 *
 *   naive : one GPU thread per pixel; every thread reads its neighbours
 *           straight from (slow) global memory, so each pixel is read
 *           up to 25 times by different threads.
 *   tiled : each block first copies its tile of the image - plus a border
 *           ("halo") of neighbour pixels - into fast on-chip shared memory,
 *           waits for the whole block (__syncthreads), then computes from there.
```

```
 *
 * Threads are arranged as a 2D grid of 2D blocks (default 16 x 16 = 256
 * threads per block), so thread (x, y) handles pixel (x, y).
 *
 * usage: ./cuda_filter input.pgm output.pgm [naive|tiled] [block_size] [repeats]
 */
#include "common.h"
#include <cuda_runtime.h>

#define CUDA_CHECK(call)                                                    \
    do {                                                                    \
        cudaError_t err_ = (call);                                          \
        if (err_ != cudaSuccess) {                                          \
            fprintf(stderr, "CUDA error: %s (%s:%d)\n",                     \
                    cudaGetErrorString(err_), __FILE__, __LINE__);          \
            exit(1);                                                        \
        }                                                                   \
    } while (0)

__constant__ int c_gauss[5] = {1, 4, 6, 4, 1};

/* ================================================== version 1: naive */

__global__ void blur_naive(const uint8_t *in, uint8_t *out, int w, int h) {
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;
    if (x >= w || y >= h) return; /* the last blocks can hang off the image */

    int sum = 0;
    for (int dy = -2; dy <= 2; dy++)
        for (int dx = -2; dx <= 2; dx++)
            sum += c_gauss[dy + 2] * c_gauss[dx + 2] *
                   in[clampi(y + dy, 0, h - 1) * w + clampi(x + dx, 0, w - 1)];
    out[y * w + x] = (uint8_t)((sum + 128) >> 8);
}

__global__ void sobel_naive(const uint8_t *in, uint8_t *out, int w, int h) {
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;
    if (x >= w || y >= h) return;

    int xl = clampi(x - 1, 0, w - 1), xr = clampi(x + 1, 0, w - 1);
    int yu = clampi(y - 1, 0, h - 1), yd = clampi(y + 1, 0, h - 1);
    int p00 = in[yu * w + xl], p01 = in[yu * w + x], p02 = in[yu * w + xr];
    int p10 = in[y * w + xl],                        p12 = in[y * w + xr];
    int p20 = in[yd * w + xl], p21 = in[yd * w + x], p22 = in[yd * w + xr];

    int gx = (p02 + 2 * p12 + p22) - (p00 + 2 * p10 + p20);
    int gy = (p20 + 2 * p21 + p22) - (p00 + 2 * p01 + p02);
    out[y * w + x] = edge_value(gx, gy);
}

/* ================================================ version 2: shared memory */

/* All threads of the block cooperatively copy a (blockDim + 2R)^2 tile into
 * shared memory. The tile is bigger than the block, so each thread may copy
 * more than one pixel (hence the strided loops). */
__device__ void load_tile(const uint8_t *in, uint8_t *tile, int w, int h, int R) {
    int tw = blockDim.x + 2 * R, th = blockDim.y + 2 * R;
    int x0 = blockIdx.x * blockDim.x - R; /* image coords of the tile's top-left */
    int y0 = blockIdx.y * blockDim.y - R;
    for (int ty = threadIdx.y; ty < th; ty += blockDim.y)
        for (int tx = threadIdx.x; tx < tw; tx += blockDim.x)
            tile[ty * tw + tx] = in[clampi(y0 + ty, 0, h - 1) * w + clampi(x0 + tx, 0, w - 1)];
    __syncthreads(); /* nobody computes until the whole tile is loaded */
}

__global__ void blur_tiled(const uint8_t *in, uint8_t *out, int w, int h) {
    extern __shared__ uint8_t tile[]; /* size given at kernel launch */
```

```
    const int R = 2;
    load_tile(in, tile, w, h, R);

    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;
    if (x >= w || y >= h) return; /* only after __syncthreads() */

    int tw = blockDim.x + 2 * R;
    const uint8_t *t = &tile[threadIdx.y * tw + threadIdx.x]; /* top-left of my 5x5 window */
    int sum = 0;
    for (int dy = 0; dy < 5; dy++)
        for (int dx = 0; dx < 5; dx++)
            sum += c_gauss[dy] * c_gauss[dx] * t[dy * tw + dx];
    out[y * w + x] = (uint8_t)((sum + 128) >> 8);
}

__global__ void sobel_tiled(const uint8_t *in, uint8_t *out, int w, int h) {
    extern __shared__ uint8_t tile[];
    const int R = 1;
    load_tile(in, tile, w, h, R);

    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;
    if (x >= w || y >= h) return;

    int tw = blockDim.x + 2 * R;
    const uint8_t *t = &tile[threadIdx.y * tw + threadIdx.x]; /* top-left of my 3x3 window */
    int p00 = t[0],          p01 = t[1],          p02 = t[2];
    int p10 = t[tw],                              p12 = t[tw + 2];
    int p20 = t[2 * tw],     p21 = t[2 * tw + 1], p22 = t[2 * tw + 2];

    int gx = (p02 + 2 * p12 + p22) - (p00 + 2 * p10 + p20);
    int gy = (p20 + 2 * p21 + p22) - (p00 + 2 * p01 + p02);
    out[y * w + x] = edge_value(gx, gy);
}

/* ============================================================== host */

int main(int argc, char **argv) {
    if (argc < 3) {
        fprintf(stderr, "usage: %s input.pgm output.pgm [naive|tiled] [block_size] [repeats]\n",
argv[0]);
        return 1;
    }
    const char *mode = argc > 3 ? argv[3] : "tiled";
    int bs = argc > 4 ? atoi(argv[4]) : 16;
    int reps = argc > 5 ? atoi(argv[5]) : 5;
    bool tiled = strcmp(mode, "tiled") == 0;
    if (!tiled && strcmp(mode, "naive") != 0) die("mode must be 'naive' or 'tiled'");
    if (bs < 1 || bs > 32) die("block size must be 1..32 (at most 32x32 = 1024 threads per
block)");
    if (reps < 1) reps = 1;

    Image img = read_pgm(argv[1]);
    int w = img.width, h = img.height;
    size_t n = (size_t)w * h;

    cudaDeviceProp prop;
    CUDA_CHECK(cudaGetDeviceProperties(&prop, 0));
    fprintf(stderr, "GPU: %s (compute capability %d.%d, %d SMs)\n",
            prop.name, prop.major, prop.minor, prop.multiProcessorCount);

    uint8_t *d_in, *d_blurred, *d_edges;
    CUDA_CHECK(cudaMalloc(&d_in, n));
    CUDA_CHECK(cudaMalloc(&d_blurred, n));
    CUDA_CHECK(cudaMalloc(&d_edges, n));
    uint8_t *edges = (uint8_t *)xmalloc(n);

    dim3 block(bs, bs);
```

```
    dim3 grid((w + bs - 1) / bs, (h + bs - 1) / bs); /* enough blocks to cover the image */
    size_t smem_blur = (size_t)(bs + 4) * (bs + 4);  /* tile + 2-pixel halo */
    size_t smem_sobel = (size_t)(bs + 2) * (bs + 2); /* tile + 1-pixel halo */

    /* GPU-side timers: ev[0] copy-in ev[1] blur ev[2] sobel ev[3] copy-out ev[4] */
    cudaEvent_t ev[5];
    for (int i = 0; i < 5; i++) CUDA_CHECK(cudaEventCreate(&ev[i]));

    float best[4] = {0, 0, 0, 0};
    double best_total = 1e30;
    for (int r = 0; r < reps; r++) { /* the first run also pays one-time GPU setup */
        CUDA_CHECK(cudaEventRecord(ev[0]));
        CUDA_CHECK(cudaMemcpy(d_in, img.data, n, cudaMemcpyHostToDevice));
        CUDA_CHECK(cudaEventRecord(ev[1]));
        if (tiled) blur_tiled<<<grid, block, smem_blur>>>(d_in, d_blurred, w, h);
        else       blur_naive<<<grid, block>>>(d_in, d_blurred, w, h);
        CUDA_CHECK(cudaGetLastError());
        CUDA_CHECK(cudaEventRecord(ev[2]));
        if (tiled) sobel_tiled<<<grid, block, smem_sobel>>>(d_blurred, d_edges, w, h);
        else       sobel_naive<<<grid, block>>>(d_blurred, d_edges, w, h);
        CUDA_CHECK(cudaGetLastError());
        CUDA_CHECK(cudaEventRecord(ev[3]));
        CUDA_CHECK(cudaMemcpy(edges, d_edges, n, cudaMemcpyDeviceToHost));
        CUDA_CHECK(cudaEventRecord(ev[4]));
        CUDA_CHECK(cudaEventSynchronize(ev[4]));

        float t[4];
        for (int i = 0; i < 4; i++) CUDA_CHECK(cudaEventElapsedTime(&t[i], ev[i], ev[i + 1]));
        double total = t[0] + t[1] + t[2] + t[3];
        if (total < best_total) {
            best_total = total;
            for (int i = 0; i < 4; i++) best[i] = t[i];
        }
    }

    write_pgm(argv[2], edges, w, h);
    char label[32];
    snprintf(label, sizeof label, "cuda_%s", mode);
    /* blur kernel, sobel kernel, and the two copies counted as "transfer" */
    print_result(label, bs * bs, w, h, best[1], best[2], best[0] + best[3]);

    for (int i = 0; i < 5; i++) cudaEventDestroy(ev[i]);
    cudaFree(d_in);
    cudaFree(d_blurred);
    cudaFree(d_edges);
    free(img.data);
    free(edges);
    return 0;
}
```

8.5 Build, Benchmark and Verification

A Makefile builds the sequential and OpenMP versions with a single command and the CUDA version with `make cuda`. The benchmark script runs every version on every image size: the sequential version, the OpenMP version with 1, 2, 4, … threads under both static and dynamic scheduling, and both CUDA versions, and it saves all timings to a CSV file. The verification function then compares every output image with the sequential output.

Code: Makefile

```
# make            -> serial + openmp (CPU versions)
# make cuda       -> cuda_filter (needs nvcc, e.g. on Google Colab)
```

```
# make clean

CC      ?= cc
CFLAGS  ?= -O2 -std=gnu11 -Wall -Wextra
NVCC    ?= nvcc
# sm_75 = Tesla T4 (Colab's free GPU); newer GPUs still run it.
NVFLAGS ?= -O2 -arch=sm_75

ifeq ($(shell uname),Darwin)
  # Apple clang needs Homebrew's OpenMP runtime: brew install libomp
  LIBOMP   := $(shell brew --prefix libomp 2>/dev/null)
  OMPFLAGS := -Xpreprocessor -fopenmp -I$(LIBOMP)/include
  OMPLIBS  := -L$(LIBOMP)/lib -lomp
else
  OMPFLAGS := -fopenmp
  OMPLIBS  := -fopenmp
endif

all: serial openmp

serial: serial.c common.h
        $(CC) $(CFLAGS) -o $@ serial.c -lm

openmp: openmp.c common.h
        $(CC) $(CFLAGS) $(OMPFLAGS) -o $@ openmp.c $(OMPLIBS) -lm

cuda: cuda_filter

cuda_filter: cuda.cu common.h
        $(NVCC) $(NVFLAGS) -o $@ cuda.cu

clean:
        rm -f serial openmp cuda_filter

.PHONY: all cuda clean
```

Code: bench.sh

```
#!/usr/bin/env bash
# Runs every version on every image in images/ and saves the timings as CSV.
#
#   bash bench.sh cpu     serial + OpenMP (1, 2, 4, ... threads; static and dynamic)
#   bash bench.sh gpu     CUDA naive + tiled
#   bash bench.sh all     everything
#
# Optional settings:  REPS=5  THREADS="1 2 4 8"  BLOCKS="8 16 32"
set -euo pipefail
cd "$(dirname "$0")"

MODE="${1:-all}"
case "$MODE" in cpu|gpu|all) ;; *) echo "usage: bash bench.sh [cpu|gpu|all]" >&2; exit 1 ;; esac
REPS="${REPS:-5}"
BLOCKS="${BLOCKS:-16}"

# Thread counts: powers of two up to the number of cores, plus the core count.
NCPU=$(getconf _NPROCESSORS_ONLN)
if [ -z "${THREADS:-}" ]; then
  THREADS=""
  t=1
  while [ "$t" -lt "$NCPU" ]; do THREADS="$THREADS $t"; t=$((t * 2)); done
  THREADS="$THREADS $NCPU"
fi

shopt -s nullglob
images=(images/*.pgm)
if [ ${#images[@]} -eq 0 ]; then
  echo "No images in images/ - run: python3 tools.py prepare" >&2
```

```
  exit 1
fi

mkdir -p results out
CSV="results/$MODE.csv"
echo "version,threads,width,height,blur_ms,sobel_ms,transfer_ms,total_ms" > "$CSV"

for img in "${images[@]}"; do
  name=$(basename "$img" .pgm)
  echo "=== $name" >&2
  if [ "$MODE" != gpu ]; then
    ./serial "$img" "out/${name}_serial.pgm" "$REPS" | tee -a "$CSV"
    for sched in static dynamic; do
      for t in $THREADS; do
        OMP_SCHEDULE=$sched OMP_NUM_THREADS=$t \
          ./openmp "$img" "out/${name}_openmp.pgm" "$REPS" | tee -a "$CSV"
      done
    done
  fi
  if [ "$MODE" != cpu ]; then
    for v in naive tiled; do
      for b in $BLOCKS; do
        ./cuda_filter "$img" "out/${name}_cuda_${v}.pgm" "$v" "$b" "$REPS" | tee -a "$CSV"
      done
    done
  fi
done
echo "Saved $CSV" >&2
```

Code: Output Verification (tools.py)

```
def compare(args):
    refs = sorted(glob.glob("out/*_serial.pgm"))
    if not refs:
        sys.exit("No out/*_serial.pgm yet - run: bash bench.sh cpu (or all)")
    all_ok = True
    for ref in refs:
        name = os.path.basename(ref)[: -len("_serial.pgm")]
        a = Image.open(ref)
        for other in sorted(glob.glob(f"out/{name}_*.pgm")):
            if other == ref:
                continue
            b = Image.open(other)
            if a.size != b.size:
                ok, msg = False, f"different size {b.size}"
            else:
                worst = ImageChops.difference(a, b).getextrema()[1]
                ok, msg = worst == 0, "identical" if worst == 0 else f"max pixel difference
{worst}"
            all_ok &= ok
            print(f"{'OK  ' if ok else 'FAIL'} {other:34s} vs serial: {msg}")
    print("All outputs match the serial version." if all_ok else "Some outputs differ!")
    sys.exit(0 if all_ok else 1)
```

CHAPTER 9

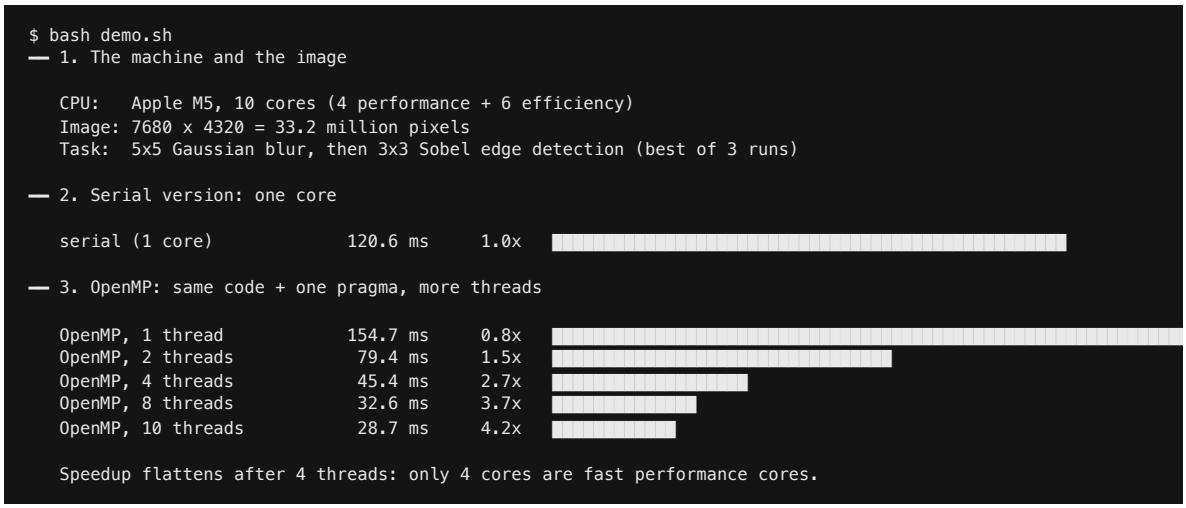
RESULTS

9.1 Experimental Setup

Parameter	Value
CPU (sequential and OpenMP)	Apple M5, 10 cores (4 performance + 6 efficiency), Apple Clang -O2 with libomp
GPU (CUDA)	NVIDIA Tesla T4 on Google Colab, 40 SMs, 2,560 CUDA cores, nvcc -O2
Image sizes	HD 1280×720, Full HD 1920×1080, 4K 3840×2160, 8K 7680×4320
OpenMP threads	1, 2, 4, 8 and 10; static and dynamic scheduling
CUDA configuration	16×16 thread blocks; naive and shared-memory kernels
Timing	Best of 5 runs; clock_gettime (CPU), CUDA events (GPU)

9.2 Program Outputs

1. Sequential and OpenMP Implementation Output



Explanation: The demonstration script runs the sequential version and then the OpenMP version with 1 to 10 threads on the 8K image.
Output: Execution time, speedup over the sequential version, and a bar proportional to the time for each configuration.

Figure 9.1: Sequential and OpenMP Implementation Output

2. OpenMP Scheduling and Correctness Output

```
── 4. Scheduling: static vs dynamic (10 threads)

   schedule(static)            35.2 ms     3.4x   ████████████
   schedule(dynamic)           29.3 ms     4.1x   ██████████

   static: equal rows per thread, fast cores wait for slow ones (load imbalance).
   dynamic: rows handed out on demand, fast cores take more.

── 5. Correctness: is the parallel output identical?

   ✓ identical   out/demo_openmp.pgm  vs  out/demo_serial.pgm
   ✓ identical   out/demo_static.pgm  vs  out/demo_serial.pgm
   ✓ identical   out/demo_dynamic.pgm  vs  out/demo_serial.pgm
```

Explanation: The same 10-thread run with static and with dynamic scheduling, followed by a byte-by-byte comparison of the outputs.
Output: Dynamic scheduling is faster, and every OpenMP output is identical to the sequential output.

Figure 9.2: OpenMP Scheduling and Correctness Output

3. CUDA Implementation Output (NVIDIA Tesla T4)

```
$ cat results/colab_gpu.csv
version,threads,width,height,blur_ms,sobel_ms,transfer_ms,total_ms
cuda_naive,256,3840,2160,0.902,0.367,3.951,5.219
cuda_tiled,256,3840,2160,0.592,0.446,3.577,4.614
cuda_naive,256,7680,4320,3.029,1.289,14.716,19.034
cuda_tiled,256,7680,4320,2.256,1.706,14.620,18.583
cuda_naive,256,1920,1080,0.337,0.134,1.085,1.556
cuda_tiled,256,1920,1080,0.142,0.095,1.024,1.260
cuda_naive,256,1280,720,0.066,0.027,0.625,0.717
cuda_tiled,256,1280,720,0.052,0.035,0.555,0.642
```

Explanation: Output of the benchmark script for the CUDA version on Google Colab: one line per kernel version and image size (threads = threads per block).
Output: Blur kernel, Sobel kernel and CPU–GPU transfer times in milliseconds, and the total.

Figure 9.3: CUDA Implementation Output

The demonstration run in Figures 9.1 and 9.2 was recorded separately from the main benchmark, so its times differ slightly (by a few per cent) from the values in Table 9.1; this is normal run-to-run variation.

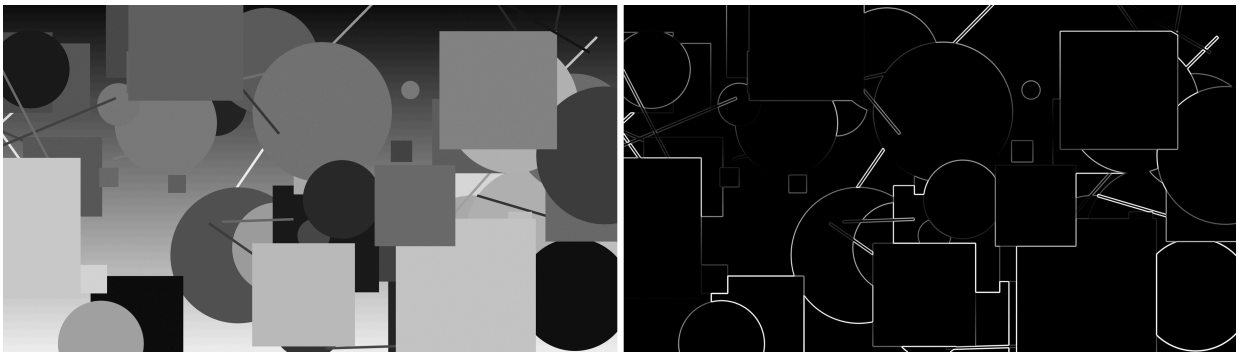


Figure 9.4: Input Image and Edge-Detection Output (Full HD)

9.3 Execution Time Comparison

Table 9.1: Execution time of all versions (best of 5 runs)

Image	Sequential (ms)	OpenMP, 10 threads (ms)	OpenMP speedup	CUDA kernels (ms)	CUDA incl. copies (ms)	CUDA speedup (kernels / incl. copies)
HD	3.8	1.0	3.8×	0.09	0.64	44× / 6×
Full HD	7.5	1.8	4.2×	0.24	1.26	32× / 6×
4K	30.6	7.6	4.0×	1.04	4.61	29× / 7×
8K	126.4	29.9	4.2×	3.96	18.58	32× / 7×

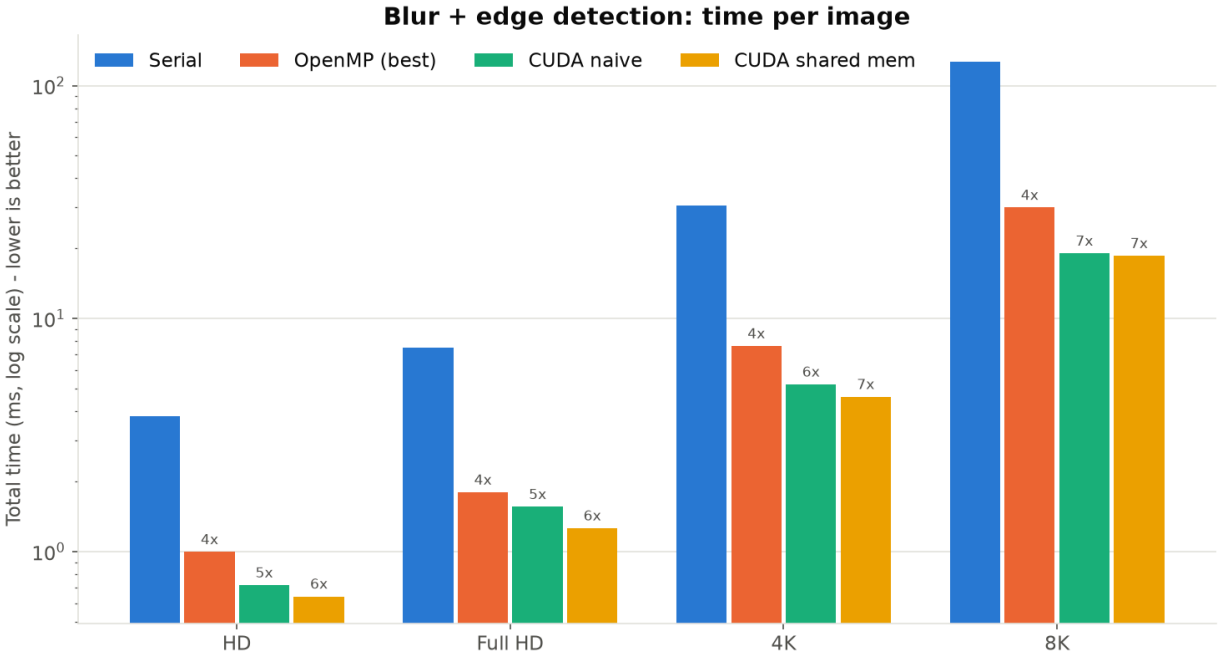


Figure 9.5: Execution Time Comparison of All Versions

Figure 9.5 compares the total time per image for all four implementations on a logarithmic scale. Every parallel version is faster than the sequential version for every image size. On the

8K image, OpenMP with 10 threads reduced the time from 126.4 ms to 29.9 ms (4.2×), and the CUDA shared-memory version took 18.6 ms including data transfer (6.8×).

9.4 OpenMP Speedup and Efficiency

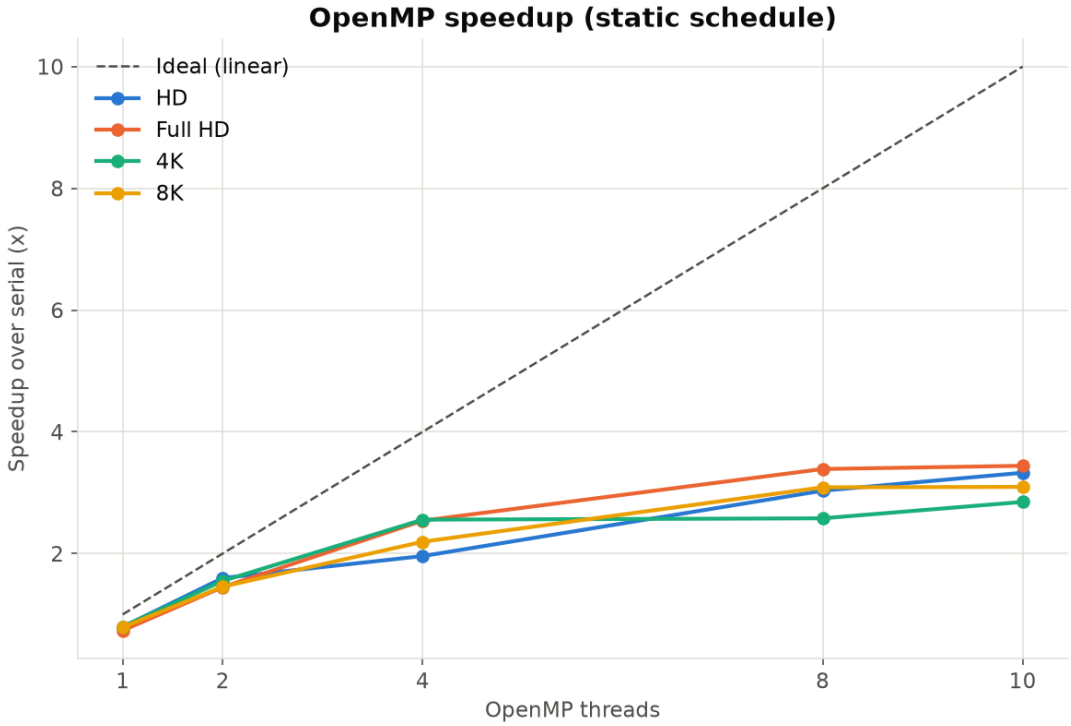


Figure 9.6: OpenMP Speedup (Static Scheduling)

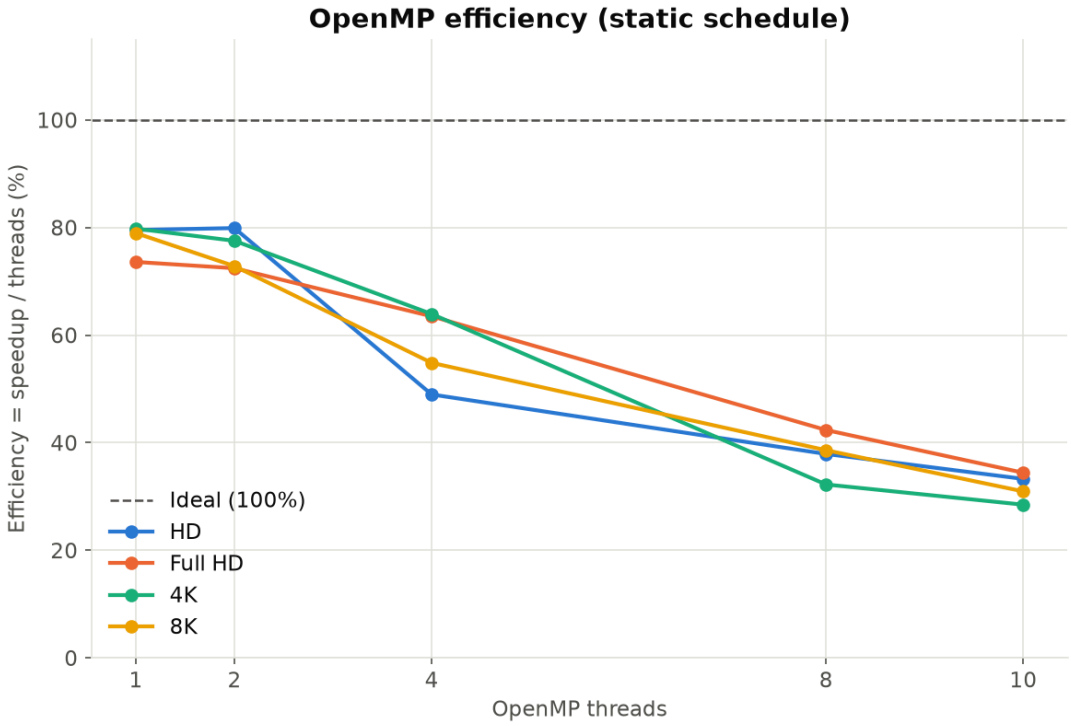


Figure 9.7: OpenMP Parallel Efficiency (Static Scheduling)

Speedup is the sequential time divided by the parallel time, and parallel efficiency is the speedup divided by the number of threads. Figures 9.6 and 9.7 show that the speedup grows well up to 4 threads and then flattens, and that the efficiency falls to roughly 30–40% at 10 threads. Three factors explain this. First, the Apple M5 has only 4 high-performance cores; threads 5 to 10 run on slower efficiency cores. Second, the filters perform little arithmetic per byte of data, so they are limited by memory bandwidth, which all threads share. Third, creating and synchronising threads has a cost: with a single thread, the OpenMP version is about 27% slower than the sequential version.

9.5 Static and Dynamic Scheduling

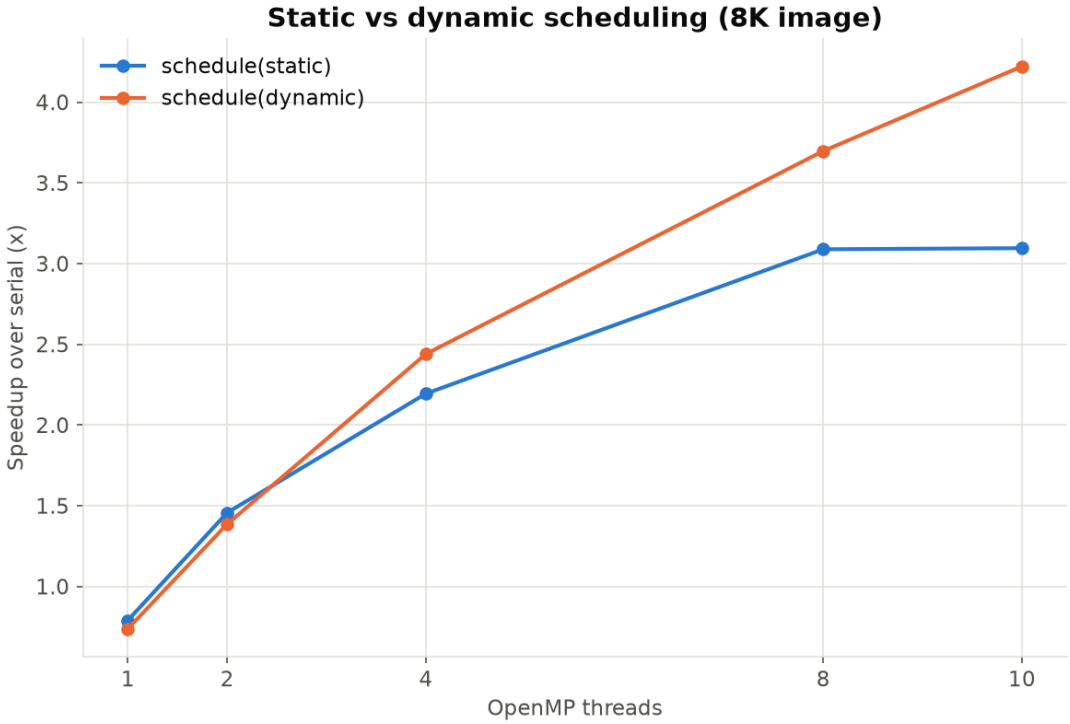


Figure 9.8: Static vs Dynamic Scheduling (8K Image)

Every image row requires the same amount of work, so static scheduling, which gives each thread an equal block of rows, might be expected to be optimal. Figure 9.8 shows that dynamic scheduling is faster on 8 and 10 threads: on the 8K image with 10 threads, the time fell from 40.8 ms (3.1×) to 29.9 ms (4.2×). The reason is that the cores are not equal. With static scheduling the fast performance cores finish their rows early and wait for the slower efficiency cores, which is a load imbalance. With dynamic scheduling, rows are handed out on demand, so the faster cores automatically process more rows.

9.6 Amdahl's Law and the Karp–Flatt Metric

Amdahl's law states that if a fraction f of the work cannot be parallelised, the speedup on p processors is S(p) = 1 / (f + (1 − f) / p). The Karp–Flatt metric uses a measured speedup to estimate this fraction experimentally: f = (1/S − 1/p) / (1 − 1/p). For the 8K image, with S = 4.2 on p = 10 threads, f ≈ 0.15. Even with an unlimited number of cores, the speedup could therefore not exceed about 1/f ≈ 6.6×. Since the code has almost no truly serial part, this measured fraction also captures the memory-bandwidth limit and the slower efficiency cores.

9.7 GPU Performance Analysis

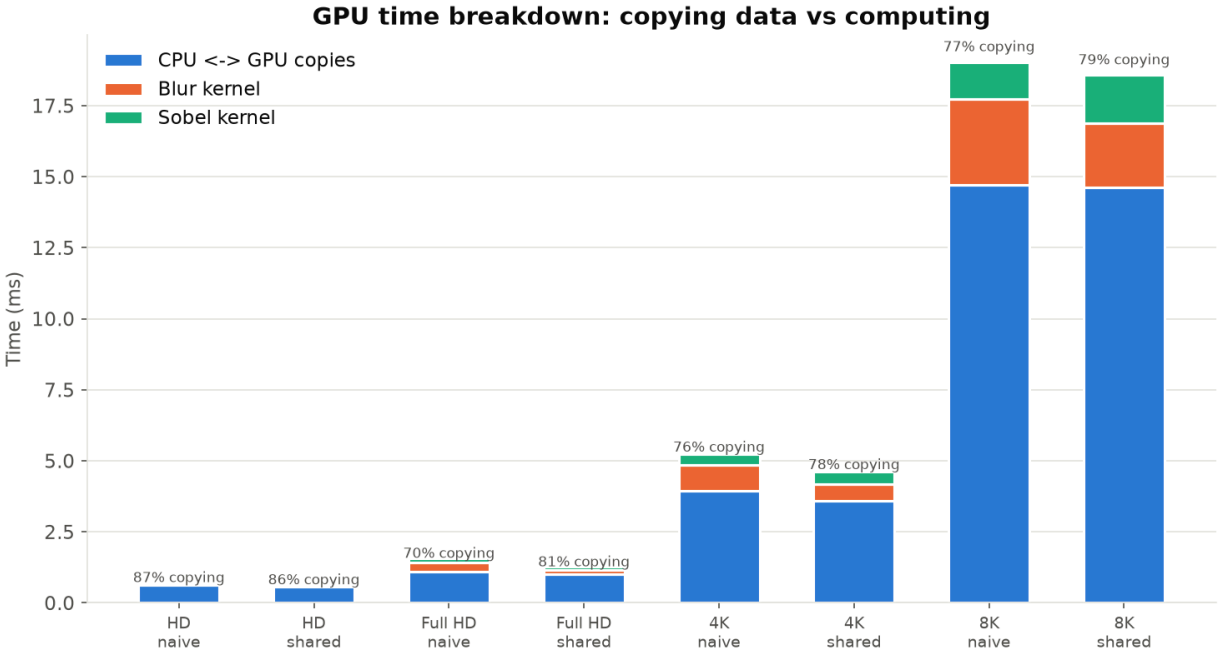


Figure 9.9: GPU Time Breakdown – Data Transfer vs Computation

Figure 9.9 divides the GPU time into data transfer and computation. On the 8K image, the two kernels take only 4.0 ms, 32 times faster than the sequential version, but copying the image to the GPU and the result back takes 14.6 ms, which is 79% of the total. Including the transfers, the overall speedup is 6.8×. This illustrates the main trade-off of heterogeneous computing: a GPU gives the greatest benefit when data remains on the GPU for many operations.

Table 9.2: Naive vs shared-memory kernels (8K image, NVIDIA Tesla T4)

Kernel	Naive (ms)	Shared memory (ms)	Change
Gaussian blur (5×5)	3.03	2.26	26% faster
Sobel filter (3×3)	1.29	1.71	32% slower

Table 9.2 shows that shared memory improved the blur kernel but slowed down the Sobel kernel. In the 5×5 blur, each input pixel is used by 25 threads, so loading it once into shared memory saves many global-memory reads. In the 3×3 Sobel filter, each pixel is used by only 9 threads, and the GPU's L1 cache already serves many of these reads, so the extra cost of copying the tile and synchronising with `__syncthreads()` outweighs the saving. An optimisation therefore pays off only when the access pattern has enough data reuse.

CHAPTER 10

ADVANTAGES

Parallel Image Edge Detection Using OpenMP and CUDA offers several advantages for improving computational performance and for understanding parallel computing techniques. The project demonstrates how the same image-processing task can be executed sequentially, on a multi-core CPU and on a GPU, and how their performance can be measured and compared.

1. **Reduced Computation Time**
   Parallel processing divides the image among many threads that run simultaneously. OpenMP reduced the processing time of an 8K image by 4.2×, and the CUDA kernels by 32×.
2. **Efficient Use of Hardware**
   OpenMP uses all the cores of a multi-core CPU, and CUDA uses the thousands of cores of a GPU, so the available hardware is fully utilised.
3. **Simple Parallelisation with OpenMP**
   The OpenMP version differs from the sequential version by a single directive per loop, which shows how easily data-parallel problems can be parallelised on shared-memory systems.
4. **Verified Correctness**
   All implementations share the same per-pixel arithmetic, and every parallel output was verified to be identical to the sequential output, pixel by pixel.
5. **Detailed Performance Analysis**
   The project measures speedup, parallel efficiency, the effect of scheduling, the serial fraction (Karp–Flatt) and the CPU–GPU transfer cost, rather than only the total time.
6. **Comparison of Two Parallel Models**
   The project compares shared-memory parallelism on a CPU (OpenMP) with massively parallel heterogeneous computing on a GPU (CUDA), including two GPU memory strategies.
7. **Scalability**
   The same code processes images from HD to 8K, and the benefit of parallel processing grows with the image size, as fixed overheads are spread over more work.
8. **Reproducibility**
   Scripts build, benchmark, verify and plot everything automatically, and a self-contained notebook runs the GPU experiments on Google Colab.

9. **Low Development Cost**

   The project uses free tools, namely GCC/Clang, OpenMP, the CUDA Toolkit, Python and Google Colab's free GPU, and requires no expensive hardware.

CHAPTER 11

APPLICATIONS

1. **Autonomous Vehicles**
   Edge detection is used to detect lane markings, road boundaries and obstacles in camera images. Parallel processing makes it possible to process high-resolution frames in real time.

2. **Medical Imaging**
   Edges outline organs, bones and tumours in X-ray, CT and MRI images. Parallel processing reduces the time needed to analyse large, high-resolution scans.

3. **Video Surveillance and Analytics**
   Security systems process continuous video streams from many cameras to detect motion and track objects; GPUs allow many streams to be processed simultaneously.

4. **Document Scanning and OCR**
   Optical character recognition systems detect the edges of characters and page boundaries before recognising text, and large batches of documents benefit from parallel processing.

5. **Industrial Inspection**
   Manufacturing lines use edge detection to find cracks, scratches and dimensional defects in products, where images must be processed at production speed.

6. **Satellite and Aerial Imaging**
   Very large satellite images are processed to detect roads, rivers, buildings and field boundaries, a natural fit for multi-core and GPU processing.

7. **Robotics**
   Robots use edge information to recognise objects and navigate, often under strict real-time limits on embedded GPUs.

8. **Photography and Video Editing**
   Sharpening, filtering and special effects in image and video editors use the same neighbourhood filters, accelerated by multi-core CPUs and GPUs.

9. **Machine Learning Pre-processing**
   Convolution, the operation behind the blur and Sobel filters, is also the core operation of convolutional neural networks, which are trained and run on GPUs.

10. **Academic Learning and Demonstration**
    The project is an educational tool for learning OpenMP, CUDA and performance analysis, with measured speedups, charts and a live demonstration script.

CHAPTER 12

CONCLUSION

The project "Parallel Image Edge Detection Using OpenMP and CUDA" demonstrates the implementation of image edge detection using sequential and parallel processing techniques. It uses C for the sequential baseline, OpenMP for shared-memory parallel processing on a multi-core CPU, and CUDA for massively parallel processing on an NVIDIA GPU.

The system implements a Gaussian blur followed by a Sobel filter in four versions: sequential, OpenMP, CUDA with global memory and CUDA with shared memory. The execution time of all versions was measured on images from HD to 8K, and every parallel output was verified to be identical to the sequential output, confirming the correctness of the implementation.

On an 8K image, OpenMP with 10 threads achieved a speedup of 4.2×, and the CUDA kernels were 32× faster than the sequential version, or 6.8× when data transfers are included. The analysis showed why the speedups are lower than ideal: the processor's heterogeneous performance and efficiency cores, shared memory bandwidth, thread overhead and, for the GPU, the cost of moving data between CPU and GPU memory. Dynamic scheduling outperformed static scheduling because it balances the load between fast and slow cores, and the Karp–Flatt metric estimated an effective serial fraction of about 0.15, which limits the speedup to about 6.6× according to Amdahl's law.

The project also showed that optimisations must match the workload: shared memory accelerated the 5×5 blur, which reuses each pixel many times, but slowed down the 3×3 Sobel filter, which has little data reuse.

Overall, the project demonstrates that parallel computing can greatly improve the performance of data-parallel image-processing tasks, and that the achievable improvement depends on the hardware, the size of the workload, the scheduling policy, the memory access pattern and the communication between processing units. Future work could overlap data transfers with computation using pinned memory and CUDA streams, use a separable blur to reduce the arithmetic, and distribute video frames across several machines using MPI.

REFERENCES

1. Pacheco, P. S. (2011). *An Introduction to Parallel Programming*. Morgan Kaufmann Publishers.

2. Pacheco, P. S., & Malensek, M. (2022). *An Introduction to Parallel Programming* (2nd ed.). Morgan Kaufmann.

3. Kirk, D. B., Hwu, W. W., & El Hajj, I. (2022). *Programming Massively Parallel Processors: A Hands-on Approach* (4th ed.). Morgan Kaufmann.

4. Gonzalez, R. C., & Woods, R. E. (2018). *Digital Image Processing* (4th ed.). Pearson.

5. Trobec, R., Slivnik, B., Bulić, P., & Robič, B. (2018). *Introduction to Parallel Computing*. Springer.

6. Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. *AFIPS Spring Joint Computer Conference*, 483–485.

7. Karp, A. H., & Flatt, H. P. (1990). Measuring parallel processor performance. *Communications of the ACM*, 33(5), 539–543.

8. OpenMP Architecture Review Board. (2024). *OpenMP Application Programming Interface, Version 6.0*. https://www.openmp.org/specifications/

9. NVIDIA Corporation. *CUDA C++ Programming Guide*. https://docs.nvidia.com/cuda/cuda-c-programming-guide/